

Assignment 2: MIPS Processor Design & Assembler

Name: Charan Deep

Roll Number: BC2025032

1. Objective

The objective of this assignment is to design a functional, 5-stage non-pipelined MIPS processor simulator and a corresponding two-pass assembler. The system translates MIPS assembly code into 32-bit machine code and simulates its execution cycle by cycle.

2. Program Implemented: Factorial Calculation

To demonstrate the functionality of the processor, a program to calculate the factorial of a number (\$n!\$) was implemented. This program was chosen because it executes a substantial loop and thoroughly tests the processor's ability to handle R-type (mul), I-type (addi, beq, sw), and J-type (j) instructions.

Assembly Logic:

The program initializes \$t0 with \$n=5\$ and \$t1 with \$1\$. It enters a loop, checking if \$t0 == 0\$ using beq. If not, it multiplies \$t1 by \$t0, decrements \$t0, and jumps back to the start of the loop. Once \$t0 reaches \$0\$, the loop terminates, and the final factorial result (\$5! = 120\$) is stored in Data Memory at address 0 using the sw instruction.

3. Code Architecture & Explanation

The project is divided into two primary Python modules:

- **assembler.py (Two-Pass Assembler):**
 - **Pass 1:** Scans the assembly file to calculate the exact byte offsets for all branch and jump labels (e.g., LOOP, DONE), removing the need for hardcoded addresses.
 - **Pass 2:** Parses each instruction, referencing a dictionary of standard MIPS opcodes and register mappings to generate the correct 32-bit binary strings. It handles PC-relative addressing for branches and absolute addressing for jumps.
- **mips.py (Non-Pipelined Processor Simulator):**
 - Reads the binary file into Instruction Memory.
 - Executes a continuous while loop where each iteration represents one full clock cycle.

- During a single cycle, the instruction passes sequentially through all five simulated architectural stages: **IF**, **ID**, **EX**, **MEM**, and **WB**.
- Outputs the active state of the Program Counter (PC), the specific actions taken in each of the 5 stages, and the values of the active registers.

4. Processor Assumptions & Design Constraints

To implement the simulator efficiently within the given constraints, the following architectural assumptions were made:

- **Non-Pipelined Timing:** The processor follows a single-cycle, non-pipelined execution model. It is assumed that the clock cycle is sufficiently long to accommodate the propagation delay of all 5 stages sequentially within a single cycle. The Program Counter (PC) only updates at the very end of the WB stage.
- **Memory Layout (Harvard Architecture):** Instruction memory and Data memory are implemented as two separate arrays. Data Memory is implemented as a limited-size array (256 slots) to optimize simulation speed.
- **Register File:** The simulator maintains a strict 32-register array. Register 0 (\$zero) is hardwired to 0 and cannot be overwritten.
- **Immediate Values:** 16-bit immediate values extracted during the ID stage are appropriately sign-extended for branch offset calculations and ALU arithmetic.

5. Source Codes

File: assembly.txt

```
lw $t0, 4($zero)
addi $t1, $zero, 1
LOOP:
beq $t0, $zero, DONE
mul $t1, $t1, $t0
addi $t0, $t0, -1
j LOOP
DONE:
sw $t1, 0($zero)
```

File: C code

```
#include <stdio.h>

int main() {

    // 1. Initialization

    int n = 5;      // Maps to: lw $t0, 4($zero)

    int result = 1;  // Maps to: addi $t1, $zero, 1


    // 2. The Loop

    while (n != 0) {  // Maps to: beq $t0, $zero, DONE

        result = result * n; // Maps to: mul $t1, $t1, $t0

        n = n - 1;      // Maps to: addi $t0, $t0, -1

    }                // Maps to: j LOOP (implicitly goes back to 'while')


    // 3. Completion

    // Maps to: sw $t1, 0($zero)

    // (Instead of raw memory storage, we print it in C)

    printf("Final Factorial Result: %d\n", result);


    return 0;
```

}

[File: assembler.py](#)

Python

```
import re

OPCODES = {'addi': '001000', 'beq': '000100', 'j': '000010', 'sw': '101011', 'lw': '100011', 'mul': '011100'}
REG_MAP = {'zero': 0, 't0': 8, 't1': 9}
REG = {f"${k}": format(v, '05b') for k, v in REG_MAP.items()}

def assemble():
    # CHANGED: Now points to your roll number file
    with open('BC2025032_assembly.txt', 'r') as f:
        lines = [line.split('#')[0].strip() for line in f]

    labels = {}
    instructions = []
    addr = 0
    for line in lines:
        if not line: continue
        if line.endswith(':'):
            labels[line[:-1]] = addr
        else:
            instructions.append((addr, line.replace(',', '')))
            addr += 4

    machine_code = []
    for pc, line in instructions:
        parts = line.split()
        cmd = parts[0]

        if cmd == 'addi':
            imm = int(parts[3]) & 0xFFFF
            binary = OPCODES[cmd] + REG[parts[2]] + REG[parts[1]] + format(imm, '016b')
        elif cmd == 'mul':
            binary = OPCODES[cmd] + REG[parts[2]] + REG[parts[3]] + REG[parts[1]] + "00000" + "000010"
        elif cmd == 'beq':
            target = labels[parts[3]]
```

```

        offset = ((target - (pc + 4)) // 4) & 0xFFFF
        binary = OPCODES[cmd] + REG[parts[1]] + REG[parts[2]] + format(offset, '016b')

    elif cmd == 'j':
        target = labels[parts[1]]
        binary = OPCODES[cmd] + format(target // 4, '026b')

    elif cmd in ['sw', 'lw']: # CHANGED: Added LW logic
        rt = parts[1]
        match = re.match(r'(-?\d+)\((\${w+})\)', parts[2])
        imm, rs = int(match.group(1)) & 0xFFFF, match.group(2)
        binary = OPCODES[cmd] + REG[rs] + REG[rt] + format(imm, '016b')

    machine_code.append(binary)

# CHANGED: Now outputs to your roll number file
with open('BC2025032_binary.txt', 'w') as f:
    f.write("\n".join(machine_code))

print("Assembler: BC2025032_binary.txt generated successfully.")

if __name__ == "__main__":
    assemble()

```

[File: mips.py](#)

Python

```
class MIPSProcessor:
    def __init__(self, binary_file):
        with open(binary_file, 'r') as f:
            self.imem = [line.strip() for line in f if line.strip()]

        self.regs = [0] * 32
        self.dmem = [0] * 256
        self.pc = 0
        self.cycle = 1
        self.used_regs = {0, 8, 9}

        # CHANGED: Pre-load the data memory at address 4 with n=5 so LW has something to read
        self.dmem[4] = 5

    def run(self):
        while (self.pc >> 2) < len(self.imem):
            print(f"\n{'='*40}\nCYCLE {self.cycle} | PC: {self.pc}")

            instr = self.imem[self.pc >> 2]
            op = instr[0:6]
            rs = int(instr[6:11], 2)
            rt = int(instr[11:16], 2)
            rd = int(instr[16:21], 2)
            imm = int(instr[16:32], 2)
            if imm > 32767: imm -= 65536

            print(f"[IF] Fetched Binary: {instr}")
            print(f"[ID] Opcode: {op} | rs: R{rs} | rt: R{rt} | imm: {imm}")

            alu_res = 0
            next_pc = self.pc + 4

            if op == '001000':
                alu_res = self.regs[rs] + imm
                print(f"[EX] ALU Add: R{rs}({self.regs[rs]}) + {imm} = {alu_res}")
```

```

elif op == '011100':
    alu_res = self.regs[rs] * self.regs[rt]
    print(f"[EX] ALU Multiply: R{rs}({self.regs[rs]}) * R{rt}({self.regs[rt]}) = {alu_res}")
elif op == '000100':
    is_equal = (self.regs[rs] == self.regs[rt])
    if is_equal: next_pc = self.pc + 4 + (imm << 2)
    print(f"[EX] Branch Check: R{rs}({self.regs[rs]}) == R{rt}({self.regs[rt]}) -> {is_equal}")
elif op == '000010':
    next_pc = (int(instr[6:32], 2) << 2)
    print(f"[EX] Jump Target Computed: {next_pc}")
elif op in ['101011', '100011']: # CHANGED: SW and LW use same address calc
    alu_res = self.regs[rs] + imm
    print(f"[EX] Address Calc: R{rs}({self.regs[rs]}) + {imm} = {alu_res}")

mem_out = 0 # Added for LW
if op == '101011':
    self.dmem[alu_res] = self.regs[rt]
    print(f"[MEM] Write: Mem[{alu_res}] = {self.regs[rt]}")
elif op == '100011': # CHANGED: LW memory read
    mem_out = self.dmem[alu_res]
    print(f"[MEM] Read: Mem[{alu_res}] = {mem_out}")
else:
    print(f"[MEM] No memory operation.")

if op == '001000':
    self.regs[rt] = alu_res
    print(f"[WB] Reg Write: R{rt} = {alu_res}")
elif op == '011100':
    self.regs[rd] = alu_res
    print(f"[WB] Reg Write: R{rd} = {alu_res}")
elif op == '100011': # CHANGED: LW register write
    self.regs[rt] = mem_out
    print(f"[WB] Reg Write: R{rt} = {mem_out}")
else:
    print(f"[WB] No register writeback.")

self.regs[0] = 0

```

```

print("-" * 40)

reg_map_rev = {0: '$zero', 8: '$t0', 9: '$t1'}

used_str = " | ".join([f'{reg_map_rev.get(r, f'R{r}')}: {self.regs[r]}' for r in sorted(self.used_regs)])

print(f"Register State: {used_str}")

self.pc = next_pc
self.cycle += 1

print(f"\n{'='*40}\nSimulation Complete. Final Factorial Result in Mem[0]: {self.dmem[0]}")

if __name__ == "__main__":
    # CHANGED: Now points to your roll number file
    MIPSProcessor('BC2025032_binary.txt').run()

```

Binary.txt

```

1  100011000000100000000000000000100
2  00100000000010010000000000000001
3  00010001000000000000000000000011
4  01110001001010000100100000000010
5  00100001000010001111111111111111
6  00001000000000000000000000000010
7  10101100000010010000000000000000

```


Output screenshots:

```
=====
CYCLE 24 | PC: 24
[IF]  Fetched Binary: 1010110000000100100000000000000000
[ID]  Opcode: 101011 | rs: R0 | rt: R9 | imm: 0
[EX]  Address Calc: R0(0) + 0 = 0
[MEM] Write: Mem[0] = 120
[WB]  No register writeback.
=====
Register State: $zero: 0 | $t0: 0 | $t1: 120
=====
Simulation Complete. Final Factorial Result in Mem[0]: 120
=====
```